



Security Audit

Qerra (hYBRID Launchpad)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	12
Centralisation	24
Conclusion	25
Our Methodology	26
Disclaimers	28
About Hashlock	29

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Qerra team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Qerra is a hybrid launchpad ecosystem that combines centralized finance (CeFi) and decentralized finance (DeFi) to provide seamless funding solutions for high-quality startups. Designed to support blockchain innovation, Qerra offers a secure and efficient platform for token launches, fundraising, and project acceleration. Its native token, QRA, is deployed on the Binance Smart Chain (BSC) Testnet, primarily for development and testing purposes. With a focus on bridging traditional finance with decentralized technologies, Qerra aims to streamline investment opportunities and foster the growth of emerging Web3 projects.

Project Name: Qerra

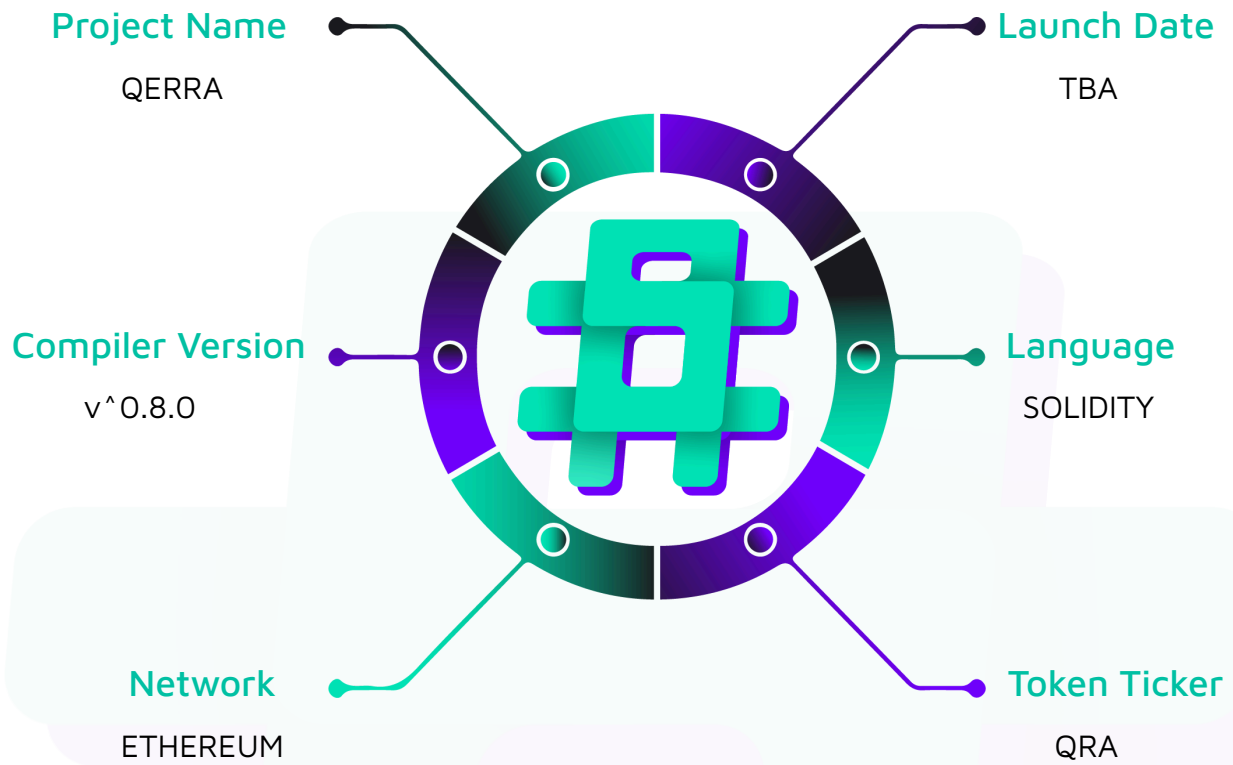
Project Type: DeFi, CeFi

Compiler Version: ^0.8.0

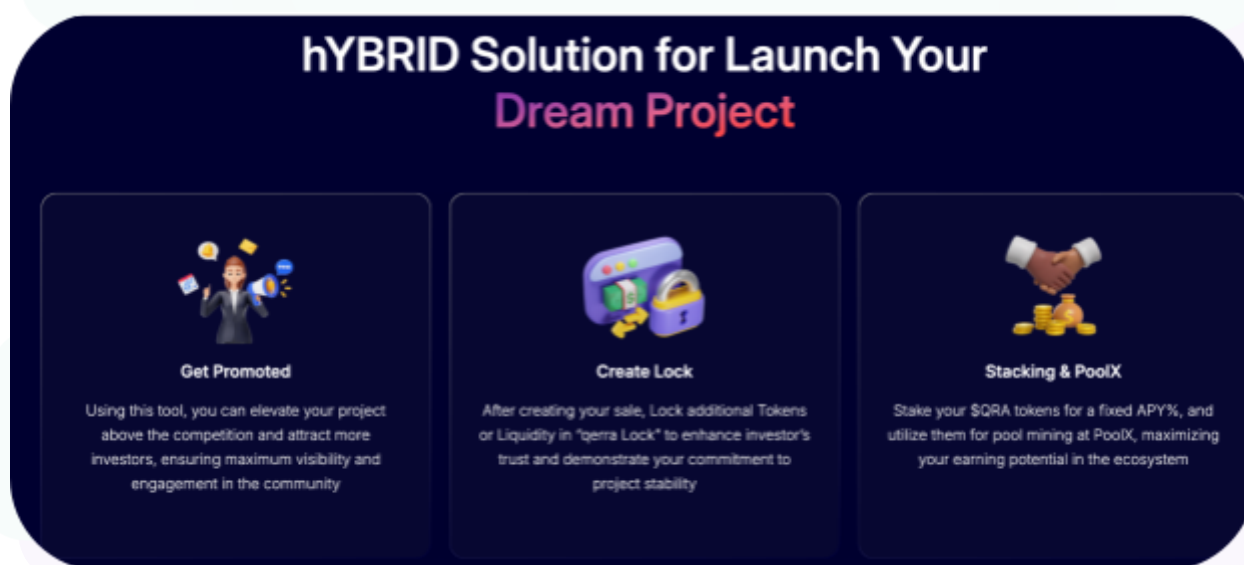
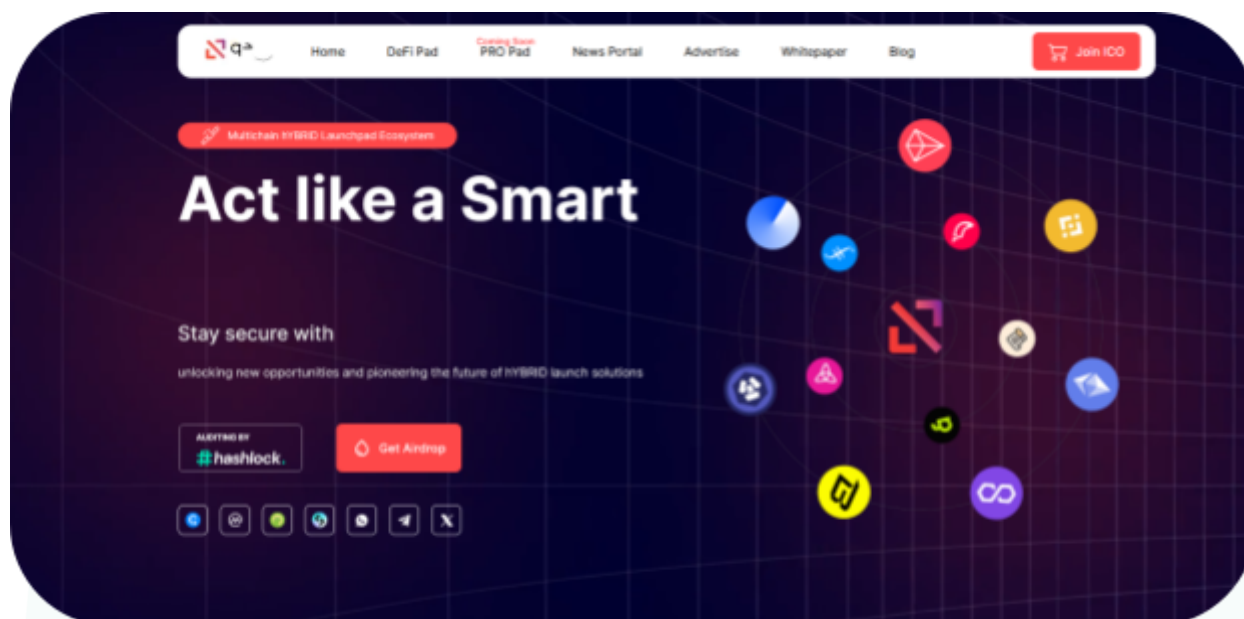
Website: <https://qerra.network>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

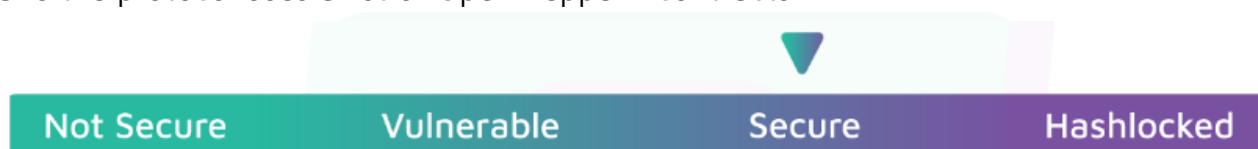
We at Hashlock audited the solidity code within the Qerra project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Qerra Smart Contracts
Platform	Ethereum / Solidity
Audit Date	April, 2025
Contract 1	Qerra.sol
Contract 1 MD5 Hash	fa99f099ce1fcf85f881495486eab430
Contract 2	QerraStakingContract.sol
Contract 2 MD5 Hash	943c3bde601af368446ff8c2fde096ad
Audited GitHub Commit Hash	
Fix Review GitHub Commit Hash	

Note: Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

We initially identified some vulnerabilities that have since been addressed.

Hashlock found:

2 High severity vulnerabilities

2 Medium severity vulnerabilities

2 Low severity vulnerabilities

2 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Qerra.sol - ERC20 token contract with integrated burn mechanisms	Contract achieves this functionality.
QerraStakingContract.sol - Staking contract meant for qerra holders - User can stake their tokens and choose the time duration of staking by choosing package - Users get rewards after staking duration is over	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Qerra project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Qerra project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] QerraToken#_update - Token burn accounting error in transfer logic

Description

The contract has burning capabilities during transfer such that 0.5 percent of tokens are being burnt. However, under the `_update` function, the contract erroneously adds the burn amount back to sender's balance.

Vulnerability Details

The vulnerability occurs in the `_update` function, which handles token transfers and burns:

```
function _update(address from, address to, uint256 actualAmount, uint256 burningAmount)
internal virtual {
    // ... input validation ...

    // VULNERABILITY: Adding back burningAmount after subtracting actualAmount
    _balances[from] = fromBalance - actualAmount + burningAmount;

    // ... recipient balance update ...

    // Separate burn operation
    if(burningAmount > 0){
        _burn(from, burningAmount);
    }

    emit Transfer(from, to, actualAmount);
}
```

Let's say a user transfers 1000 tokens. Here, the expected result is that the sender should be deducted by 1000 token and receiver should get 995 token, rest 5 being burnt.

However, the contract calculates `burningAmount` as 5 and `actualAmount` as 995. It then sets `_balances[from] = oldBalance - 995 + 5` followed by a `_burn` call of `(from, 5)`

which deducts 5 more tokens. The problem here is that contract is adding the burnt amount again.

First calculation: $\text{balance} = \text{balance} - 995 + 5$ (only deducts 990)

Second step via `_burn`: Deducts 5 more tokens

Total deduction: 995 tokens (incorrect)

Proof of Concept

Paste the following POC in your local foundry test setup.

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.13;

import {Test, console} from "forge-std/Test.sol";
import {Qerra} from "../src/qerra.sol";

contract BurnTest is Test {
    Qerra public qerra;
    address admin = address(0x123);
    address user = address(0x234);

    function setUp() public {
        qerra = new Qerra(admin);
        qerra.balanceOf(admin);
    }

    function testBurnLogicBug() public {

        uint256 transferAmount = 1000e18;
        uint256 expectedAdminDecrease = 1000e18;
        uint256 expectedUserReceive = 995e18;
        uint256 adminInitialBalance = qerra.balanceOf(admin);

        console.log("Amount to be transferred", transferAmount);
        console.log("Expected admin decrease", expectedAdminDecrease);
        console.log("expectedUserReceive", expectedUserReceive);
        console.log("Admin Initial Balance", adminInitialBalance);

        vm.prank(admin);
        qerra.transfer(user, transferAmount);

        uint256 userReceived = qerra.balanceOf(user);
        uint256 adminFinalBalance = qerra.balanceOf(admin);
        uint256 adminLeft = adminInitialBalance - adminFinalBalance;

        console.log("User Received", userReceived);
        console.log("adminleft", adminLeft);
    }
}
```

Upon seeing the logs we can clearly see the discrepancy

```
[PASS] testBurnLogicBug() (gas: 66450)
Logs:
  Amount to be transferred 1000000000000000000000
  Expected admin decrease 1000000000000000000000
  expectedUserReceive 995000000000000000000
  Admin Initial Balance 2100000000000000000000000000
  User Received 995000000000000000000
  adminleft 995000000000000000000
```

Impact

Any user that transfer token is getting more tokens than intended.

Recommendation

Decrease from balance with `actualAmount`, burn the `burningAmount` and send the rest to receiver:

```
function _update(address from, address to, uint256 actualAmount, uint256 burningAmount)
internal virtual {
    // ... input validation ...

    // FIXED: Deduct the full amount
    _balances[from] = fromBalance - actualAmount;

    // ... recipient balance update ...
```

Status

Resolved

[H-02] QerraToken#decimals - Incorrect decimal assumption will result in staking contract being useless.

Description

The current staking contract is assuming qerra token to be 8 decimal places. However, the current qerra token is 18 decimal place. This will result in incorrect calculation in staking contract which won't work.

Vulnerability Details

The QerraStaking contract works as if given token is 8 decimal place. This can be confirmed by decimal variable being 1e8.

However, the given qerra token:

<https://testnet.bscscan.com/token/0xeBCfBcB806a25a60313D1431529F811A97EAbad>

which is expected to be the token address of the staking contract is still 18 decimal.

Impact

This will result in `checkTotalSupply` reverting such that no one will be able to stake.

Recommendation

Change the decimals of qerra token from 18 to 8.

Status

Resolved

Medium

[M-01] QerraStakingContract#withdraw - Staking contract always assumes enough token balance in case of withdraw which is incorrect.

Description

The staking contract will give out additional rewards along with the staked amount once withdrawal happens. However, the contract always assumes that contract has enough tokens to cover for withdrawal at any given moment. On the contrary, there is no transfer of tokens happening during initialization.

Vulnerability Details

The current contract poses a risk of insufficient balance for any user at the time of withdrawal. There is no transfer of tokens to staking contract to cover for rewards.

```
function withdraw(uint _id) external checkProfile(_id) nonReentrant {
    require( block.timestamp - package[_id].timestamp >=
timestamp[package[_id].package - 1], "The package period is still ongoing");

    uint amount = (package[_id].stakedToken * rewardAmnt[package[_id].package - 1])
/ decimal + package[_id].stakedToken;

    package[_id].isActive = false;
    package[_id].amountWithdrawn = amount;
    require(token.transfer(msg.sender, amount), "Transfer error occurred");
}
```

Consider a scenario where there is only one user that stakes 1000 tokens. Since qerra token has an integrated burn mechanism, 0.5 percent of tokens i.e. 5 tokens, will be burnt. As a result, the contract will receive only 995 tokens in the contract.

At the time of withdrawal, the user will get more than 1000 + reward tokens depending upon the package they choose. However, at this state the contract does not have enough balance to cover for withdrawal, resulting in a revert.

Impact

Users won't be able to withdraw, and their tokens will be locked out.

Recommendation

Consider having a greater balance of tokens to cover for every withdrawal. Optionally, ensure that the owner can transfer more tokens before every package expiry to prevent the last users lock of tokens.

Status

Resolved

[M-02] QerraStakingContract#stake - Users will get more rewards than intended, resulting in loss of funds for protocol.

Description

The current contract decrease `totalSupply` via `checkTotalSupply` first followed by `transferFrom` of actual tokens. However, as explained in [\[M-01\]](#), the contract will receive less tokens than intended.

Vulnerability Details

As described above, the function assumes that the amount received will be equal to amount given by the user. However, this is not the case.

```
function stake(uint _amount, uint _package) external nonReentrant {
    require(_amount >= 1e8, "Amount must be greater than 1 qra");
    user[msg.sender].push(id);
    checkTotalSupply(_amount, _package);
    require(
        token.transferFrom(msg.sender, address(this), _amount),
        "Transfer error occurred"
    );
    // rest of the code..
}
```

A user staking 1000 tokens will get rewards for 1000 tokens however, the contract will only receive 995 tokens.

Impact

Since the overhead cost of transfer is user's responsibility, they are getting slightly more yield than the value they have transferred to the contract.

Recommendation

Integrate a fee on transfer like logic :

```
function stake(uint _amount, uint _package) external nonReentrant {
    require(_amount >= 1e8, "Amount must be greater than 1 qra");
    user[msg.sender].push(id);

    // Record balance before transfer
    uint balanceBefore = token.balanceOf(address(this));

    // Execute transfer
    require(
        token.transferFrom(msg.sender, address(this), _amount),
        "Transfer error occurred"
    );

    // Calculate actual received amount (accounting for burn)
    uint balanceAfter = token.balanceOf(address(this));
    uint actualReceived = balanceAfter - balanceBefore;

    // Calculate rewards based on actual received amount
    checkTotalSupply(actualReceived, _package);

    // Store the actual received amount in the package
    package[id] = Package(
        msg.sender,
        block.timestamp,
        block.timestamp,
        actualReceived, // Use actual received amount
        perDayInc[_package - 1] / decimal,
        0,
        _package,
        0,
        true
    );

    emit stakeEvent(msg.sender, actualReceived, _package, id);
    ++id;
}
```

Status

Resolved

Low

[L-01] QerraStakingContract#checkTotalSupply - total supply should be `<=` instead

Description

The `checkTotalSupply` function uses `<` instead of `<=` when checking if there's enough supply to fulfill a reward. This prevents users from staking when the exact remaining supply would be consumed.

```
function checkTotalSupply(uint _amount, uint _package) public {
    uint rewardAmount = (_amount * rewardAmnt[_package - 1]) / decimal;
    require(
        rewardAmount < totalSupply, //@audit could use <= here
        "There is not enough supply to fulfill this package"
    );
    totalSupply -= rewardAmount;
}
```

Impact

When the remaining supply could fulfill exactly one more stake, that stake will be rejected, effectively wasting a small portion of the allocated rewards.

Recommendation

Status

Resolved

[L-02] QerraStakingContract - Withdraw completely ignores reward calculation

Description

The staking contract implements two separate mechanisms for handling rewards that don't properly interact with each other

The `calculateReward` function tracks accumulated rewards over time in `package[_id].earningTillNow`

The `withdraw` function calculates the withdrawal amount directly without referencing these tracked rewards.

```
function withdraw(uint _id) external checkProfile(_id) nonReentrant {
    require( block.timestamp - package[_id].timestamp >=
timestamp[package[_id].package - 1], "The package period is still ongoing");

    uint amount = (package[_id].stakedToken * rewardAmnt[package[_id].package - 1])
/ decimal + package[_id].stakedToken;

    package[_id].isActive = false;
    package[_id].amountWithdrawn = amount;
    require(token.transfer(msg.sender, amount), "Transfer error occurred");
}
```

Impact

Users spend unnecessary gas fees calling `calculateReward()` with no effect on their final payout

This removes the importance of calculate rewards. Even if a user does not call this, they are expected to get full amount with rewards back.

Recommendation

Simplify by removing `calculateRewards`

Status

Resolved

[L-03] QerraStakingContract#stake - Add a check to ensure package does not go out of bounds

Description

The staking contract does not validate that the `_package` parameter is within valid bounds (1-6) in the `stake`.

Impact

Revert in case of package is out of bounds.

Recommendation

Add a check to ensure it stays within bounds

```
function stake(uint _amount, uint _package) external nonReentrant {  
    require(_amount >= 1e8, "Amount must be greater than 1 qra");  
    require(_package >= 1 && _package <= 6, "Invalid package: must be between 1-6");  
  
    // Rest of the function remains the same  
    // ...  
}
```

Status

Resolved

QA

[Q-01] Contracts - Use Ownable2Step instead

Description

The contract uses the single-step Ownable pattern for ownership management, which transfers ownership immediately upon calling `transferOwnership()`. This creates a risk that ownership could be accidentally transferred to an incorrect or inaccessible address, permanently locking admin functions.

Impact

Risk of permanently losing contract ownership if transferred to an incorrect address

Recommendation

Replace the current Ownable implementation with OpenZeppelin's Ownable2Step, which implements a two-step ownership transfer

Status

Resolved

[Q-02] QerraToken -Make constant more readable

Description

The constructor of the Qerra token contains a hardcoded token supply value that lacks readability and could be improved for better code quality:

Recommendation

Define as a named constant

```
uint256 private constant INITIAL_SUPPLY = 210_000_000;  
_mint(initialOwner, INITIAL_SUPPLY * 10 ** decimals());
```

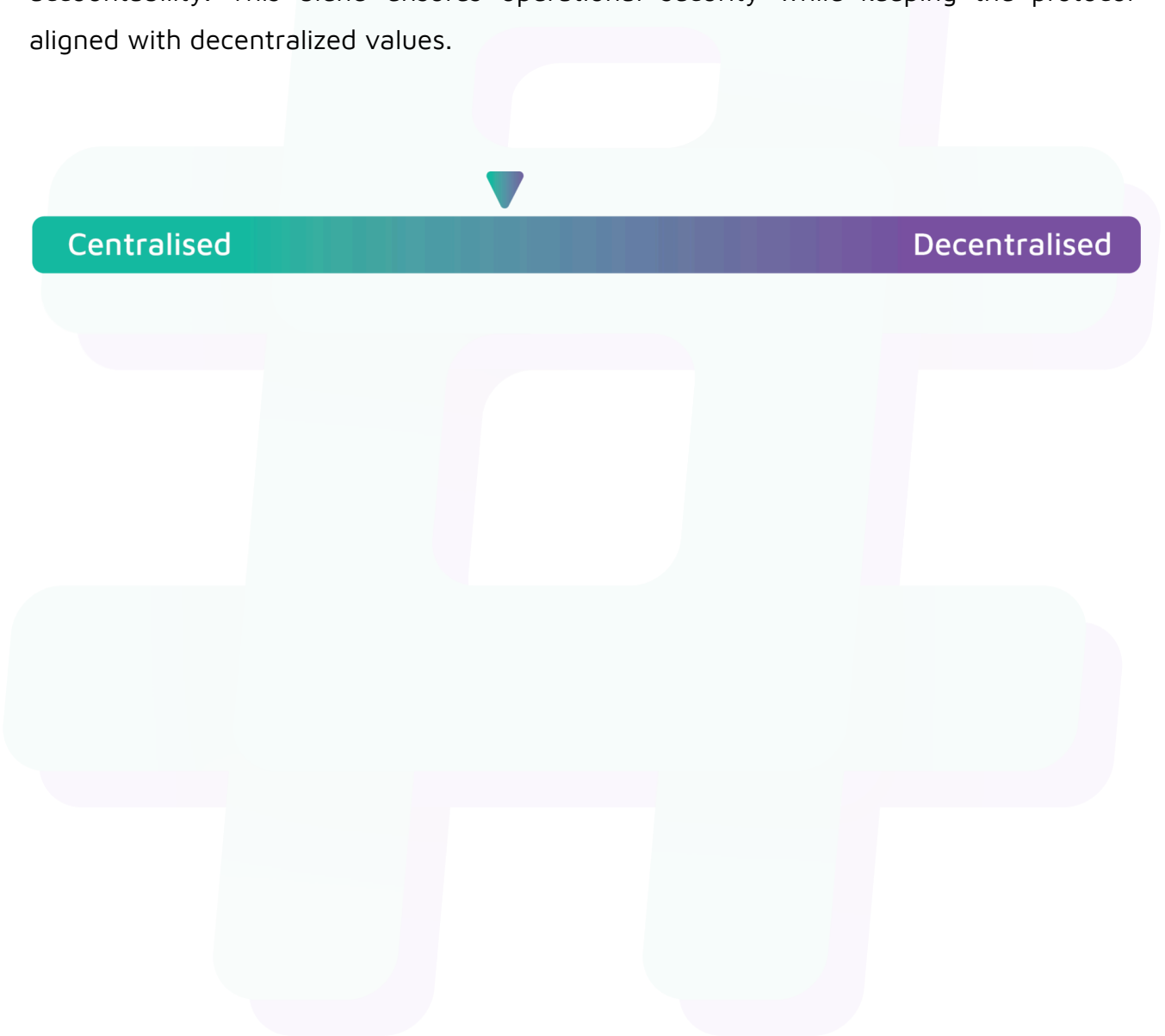
Status

Resolved

Centralisation

The Qerra project values community-centered and transparency over centralisation.

The Qerra project emphasizes a community-centric approach while maintaining transparency. Although some critical functions remain centralized for enhanced security and functionality, the project strives to balance decentralization with internal team accountability. This blend ensures operational security while keeping the protocol aligned with decentralized values.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Qerra project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.